

Class 5 Assignment: Personal Wiki with Local Gemma + RAG

Build your own CLI and harness to talk to your own data, using a local open-weight model that fits your computer and works offline.

Overview

Build a personal wiki from your own notes and documents. Use local Gemma to organize linked pages and power three distinct CLI modes: chat as a personal assistant, ask for grounded factual answers, and search for original passages. Build your own harness to connect the interface, conversation, retrieval, prompts, model, citations, and saved outputs. This is a standalone, one-and-done Assignment 4, separate from the capstone. No sample GitHub repository is provided.

Model, Retrieval, RAG, and Harness

Model: local Gemma generates text from the instructions and context your program supplies. It does not automatically read your files, remember every conversation, or operate your tools.

Retrieval tool: code that searches your local index and returns relevant original passages with source paths. Its job is to find evidence. Search mode exposes this tool's results directly, without generating an answer.

RAG workflow: retrieve relevant passages, include them in a prompt, and have the model answer using that evidence. Ask mode uses this workflow. RAG supplies context at answer time; it does not train the model.

Harness: the surrounding application code you build. It manages modes, instructions and personality, conversation context, tool calls, prompt assembly, local model calls, citation checks, errors, and saved outputs. Retrieval is one tool the harness can use; the harness decides when it is needed.

CLI: the terminal interface through which a user controls the harness. You can reuse inference and search libraries, but you must understand and assemble the connecting code. Explain one path from a user's command to the displayed result.

Three Required Modes: Chat, Ask, Search

Chat = personal assistant. Give it a recognizable voice and a clear description of what it can actually do. It should help brainstorm, draft, plan, and work through ideas, using recent conversation for follow-ups. Retrieve notes when the request needs them, not for every message. Cite claims based on retrieved notes; label proposals as suggestions and never invent personal facts.

Chat example: "what can you help me with?" should explain the assistant's real capabilities and suggest a useful starting point. It should not return "insufficient evidence" or pull in unrelated course passages. After a draft or plan, "make that shorter" should use the conversation.

Ask = factual personal research. Treat each question independently of chat history. Retrieve relevant evidence, answer directly in a neutral voice, and show citations. If the sources do not support the answer, say so. Personality, brainstorming, and guesses must not replace evidence.

Search = inspect the sources. Return matching original passages and source locations. Do not generate a synthesized answer. This lets you check whether retrieval found the right material before evaluating the model.

The harness enforces these differences through separate instructions, context, and tool use.

Chat history is conversation context, not automatically verified source material. If you add a save-memory feature, make saving an explicit user action and keep generated drafts separate from original evidence.

Chat, ask, and search describe interaction modes. Local and online describe where the model runs. All three interaction modes are required locally; an online option is an extension and never replaces offline operation.

What You Are Submitting

- Your own CLI and harness source code, with distinct chat, ask, and search modes, plus ingestion and help commands. A notebook or web interface is an optional extra.
- At least three original sources in vault/raw/, preserved unchanged, plus linked wiki pages in vault/wiki/ and a current vault/index.md. Include Obsidian screenshots of your personal memory: an open wiki note with source references, the vault's page list or index, and the graph view showing connections.
- Three answerable questions tested in ask mode, with expected sources, retrieved passages, actual Gemma answers, and citations.
- One question your sources cannot answer, tested in ask mode, with the actual response showing insufficient evidence. Also include the short chat/search mode checks below.
- A short explanation of your model, data, retrieval, and prompt choices, plus one observed limitation and a proposed improvement.
- A terminal recording or screenshots and saved evidence cards showing CLI ingestion, all four ask-mode question tests, and the chat/search mode checks running with the internet disconnected.
- A README with exact setup and CLI commands, model/version and quantization, runtime, device specs, model-choice rationale, measured memory use and response time, and links to evidence. Online mode is optional.

How Your Assignment Is Graded

This assignment uses the course framework: deliverable quality (4 points), testing & evaluation (3 points), and working result (3 points), for 10 points total.

- Deliverable quality (4 points): readable harness code; a wiki that is usable in Obsidian, with short descriptive filenames, topic navigation, readable graph labels, and meaningful working links; traceable sources; clear setup instructions; and an explanation of how your own data reaches the model.
- Testing & evaluation (3 points): three answerable ask questions and one unsupported ask question, with expected evidence, retrieved passages, actual answers, checked citations, and proof of offline execution. Include the chat/search mode checks. Explain failures honestly.
- Working result (3 points): demonstrate local Gemma, wiki ingestion, and usable chat, ask, and search modes working through your own harness without internet access.

Partial credit follows the evidence provided. Missing offline proof, an untested retrieval step, or a nonworking CLI affects the relevant category. A cloud-only service or ready-made document-chat interface does not satisfy the required local CLI and harness.

Model size, website polish, and optional online deployment do not replace the required workflow or evidence. Explain limitations instead of hiding weak answers.

Make the README the entry point: link your CLI code, wiki, setup and command examples, four ask-mode evidence cards, chat/search mode checks, and offline demonstration.

Set Up Your Local Model

- Start with the official Gemma documentation. Choose an open-weight Gemma model and a quantized version that your computer can run.
- Install a compatible local inference runtime. Existing runtimes and libraries are allowed; you write the code that connects the model to your data.
- While online, download the model weights, packages, tokenizer, and any embedding model. Confirm they are stored locally before the offline test.
- Record the exact model identifier, version, quantization, runtime version, and device specs. Explain why this model fits your computer. No model training or fine-tuning is required.

Choose Gemma for Your Computer

B means billion parameters. Google's E2B and E4B names describe effective parameter counts; their extra embedding tables also use memory. Quantization stores weights with fewer bits to reduce memory use, with possible quality tradeoffs.

E2B (often called 2B): approximately 2.9 GB to load at Q4_0. Start here when memory is tight.

E4B (often called 4B): approximately 4.5 GB to load at Q4_0. Try it when your device has room for a larger model.

26B A4B MoE: approximately 14.4 GB to load at Q4_0. It activates about 4B parameters per token, but fast inference still loads all 26B weights. It does not have a 4B memory footprint.

These official Gemma loading estimates are not total application requirements. Q4_0 is a 4-bit format. Leave additional memory for context, the runtime, retrieval, the operating system, and other apps. Check the exact model file and runtime you choose.

RAM is system memory; dedicated VRAM belongs to a discrete GPU. On Apple Silicon, CPU and GPU share unified memory. Disk space stores downloads; it is not working memory. CPU or partial GPU offloading can be slower than keeping the model in GPU memory.

Practical starting points, not guaranteed minimums: on an 8 GB Apple Silicon Mac, start with quantized E2B and short context; on a 16 GB Mac, consider E2B or E4B. On a PC with 6–8 GB dedicated VRAM, start with E2B and try E4B only if the runtime and context fit. With 32 GB or more unified memory, or 24 GB or more dedicated VRAM, 26B MoE is a candidate, not a requirement.

Check OS/chip support before installing your runtime. For CPU-only use, start with E2B in a compatible runtime and measure speed. Leave memory available for the rest of your computer; nominal installed memory is not all free.

Run ingestion and one RAG answer on your actual device. Record memory use and response time, then justify the smallest model that works for your wiki. You do not need to test all three sizes, buy hardware, or choose the largest model.

Sources: [official Gemma model and memory documentation](#); [LM Studio system requirements](#).

The device starting points above are classroom guidance; verify your chosen runtime and model on your own computer.

Build Your Personal Wiki

- Choose at least three sources you can inspect, such as course notes, research articles, or personal project notes. Use material you have permission to include in your submission.
- Keep original sources unchanged in vault/raw/. Store generated summaries and linked pages separately in vault/wiki/.
- Have your harness read a source, send its text and instructions to local Gemma, and save a reviewed wiki page with a reference back to the source.
- Connect related pages with links and update vault/index.md so a reader and your retrieval code can find them.
- Plain text and Markdown are sufficient. If you support PDFs, check extraction quality; any OCR or parsing needed during the offline demonstration must also run locally.
- Split long text into useful passages and retain the source path and page or section when available. Explain how much text your harness passes to Gemma.
- Open your generated personal memory vault in Obsidian and check that links and source references work. Capture an open wiki note with source references, the page list or index, and the graph view showing connections. These Obsidian screenshots are required evidence; the files remain ordinary Markdown.
- Review generated summaries against the originals. Correct invented or misleading statements in the wiki rather than changing the evidence.

Make the Wiki Readable in Obsidian

Your personal memory has two audiences: you browsing in Obsidian and the LLM retrieving evidence. Both must be able to use it. Readable note names, navigation, and meaningful connections are part of the deliverable.

Name notes for their subject. Use short, natural filenames, usually 2–6 words, such as GPU Parallel Training.md, Course Project Ideas.md, or Retrieval Augmented Generation.md. Match the note's first heading to its filename. Do not use exported task titles, full sentences, timestamps, UUIDs, hashes, or chunk numbers as generated note names. Keep machine IDs and original filenames in properties or a source catalog. An alias or a prettier link label alone does not satisfy this requirement: the actual filename and graph label must be readable.

Example: replace

class4-gpu-parallel-training-visual--task-1-gpu-parallel-training-slide-visual--c8d92e24fd.md with GPU Parallel Training.md. Preserve the old identifier in metadata. If two topics need the same name, add a short meaningful qualifier, such as Memory - Computers.md and Memory - Learning.md, instead of a random suffix.

Open vault/ itself as the Obsidian vault, not the entire code repository or course folder. Keep unchanged originals in raw/, reviewed notes in wiki/, and embedded images in attachments/ when needed. Organize wiki/ with a few useful topic folders, for example Concepts/, Projects/, and People/; use only categories that fit your sources. Keep code, logs, retrieval chunks, embeddings, test answers, and other generated machine files outside the vault. Keep index.md as a human landing page, grouped by topic with short descriptions and links.

Write one coherent subject per note, with a short summary, useful details, source references, and related notes. Merge overlapping notes about the same subject. A retrieval chunk is not automatically a new wiki note. Use real internal links such as `[[GPU Parallel Training]]` and explain why the linked topic is relevant. An index that links to everything is navigation, but does not replace meaningful links between related notes; do not add unrelated links just to make the graph look connected.

Make the graph useful for exploration. Show the curated wiki notes, for example with the graph filter path:wiki/, and turn off Attachments so images and PDFs do not overwhelm the view. Use topic groups or a local graph when helpful. Zoom until labels are readable. Filtering is presentation, not a substitute for fixing filenames, duplicate notes, or broken links. Keep source references accessible from each note.

Check the result in Obsidian: start at index.md, find a topic, open a related note, and follow its source reference. Confirm the page list, headings, and graph labels make sense without opening every file. Check all internal links and source references for missing or ambiguous targets. A dense graph screenshot alone is not proof of a usable wiki.

If you already built the vault, back it up and clean up the generated notes. Rename them, merge duplicates, and update incoming links, the index, source mappings, and retrieval paths.

Preserve original source contents and identifiers. Rebuild the retrieval index and rerun the existing question tests so the cleanup does not break citations. Ingest the same source again and confirm that it updates the intended notes without creating duplicates or bringing back machine-style names.

Your Three Choices

- Data: what is this wiki for, which sources belong in it, and what questions should it answer? Keep the scope small enough to verify.
- Model: which Gemma size and quantization fit your own device? Justify the choice using RAM/VRAM, runtime compatibility, measured memory use, and response time. All three model sizes can satisfy the assignment; you do not need to run all three.
- Retrieval: start with a local index and keyword search, or use local embeddings. A hosted embedding API would break the offline requirement.
- Write down your choices and expected behavior before testing. Use those expectations to interpret the actual results.
- The model has a limited context window. Select relevant passages and keep their source labels instead of sending the entire wiki for every question.
- If you change a setting after a failure, document the change and rerun the affected tests. Keep the earlier result as evidence of what improved or stayed wrong.

Evaluate Retrieval and Answers Separately

- Before building, write three answerable questions, their expected source passages, and one question for which your sources contain no answer.
- Inspect retrieval first: did the right passage appear? If it did not, revise your indexing or retrieval before blaming the model.
- Then inspect the answer: does each material claim follow from the retrieved evidence, and does its citation point to the correct source?
- A fluent answer can still be wrong. Record missing evidence, irrelevant retrieval, invented details, and unsupported citations as failures.

Run Your Four Question Tests

An eval is a fixed question, an expected behavior, and the system's actual result. Run the following four research questions in ask mode, independently of chat history. Create this small test set for your own wiki.

- Test 1: a direct question answered by one source. Save the relevant source and passage before running your harness.
- Test 2: another answerable question, ideally phrased differently from the source so you can inspect how retrieval handles wording.
- Test 3: a third question with known supporting evidence. It can connect two sources if your small corpus supports that.
- Test 4: a plausible question whose answer is missing from the wiki. The expected behavior is an explicit statement that the evidence is insufficient.
- Keep your test expectations outside the searchable wiki. The harness should retrieve the underlying source material, not a file containing your answer key.

- For each test, save the question, retrieved passages and paths, exact model identity, local/online mode, generated answer, citations, and your assessment.
- Run all four tests offline. If you add an online mode, test it separately and label its evidence so it cannot be confused with the required local run.

Check the Mode Boundaries

Chat: try "what can we do?" and "what can you help me with?" Expect an accurate explanation of capabilities, without an unnecessary notes search, unrelated citations, or an insufficient-evidence refusal.

Chat follow-up: request a short draft or plan, then say "make that shorter." Expect the assistant to use the current conversation. Any factual claims drawn from the wiki still need supporting citations.

Search and ask: search a topic and show the original passages, with no generated answer. Then run a standalone ask question with citations. Confirm that a claim made only in chat is not treated as source evidence in ask.

Save a short transcript of these checks alongside the four ask-mode evidence cards, and demonstrate the mode checks offline too. Keep the original four research questions; these checks verify the interface and harness behavior.

Example: Evidence vs. Guessing

Suppose your wiki contains a project note that says: "The workshop starts at 10 a.m. in Room 204."

Supported question: "Where is the workshop?" Retrieve that passage, ask Gemma to answer from it, and show "Room 204" with a citation to the note.

Unsupported question: "Who is catering the workshop?" If no source says, the answer should explain that the wiki does not contain that information.

Ask-mode flow: question → retrieve local passages → combine evidence and instructions → call local Gemma → display answer and citations. Chat can skip retrieval when the turn is conversational; search stops after returning passages.

A citation is not proof by itself. Open the cited source and check that it supports the claim. A correct answer based only on the model's general knowledge is not grounded RAG evidence. Keep source files, wiki pages, instructions, tests, and results in separate folders or files so your harness reads only the intended material.

Build Your Own Personal Wiki CLI

- Create a command-line interface: a user types terminal commands to run your own harness. Implement chat, ask, search, ingest, and help with the distinct behaviors defined above. The example names below are a design to build, not a supplied tool; your names may differ.

- Your harness must choose the requested behavior: chat uses conversation context and retrieves only when useful; ask retrieves evidence for a standalone factual answer; search returns matching passages without generating an answer. Show the model and execution setting; local is the default.
- Keep research rules and assistant personality/instructions explicit, for example in wiki-instructions.md and persona.md. Your harness must load the relevant instructions for each mode; the model does not automatically read project files. Give chat an accurate description of its capabilities and commands.
- Use libraries for model inference, parsing, or search as needed. A ready-made document-chat app alone is not your harness. Be able to trace one question through the code you assembled.

Example CLI Commands to Implement

wiki ingest ./vault/raw — read local sources, generate linked wiki pages, and update the local index.

wiki search "workshop location": show original matching passages and source paths. Do not generate an answer; search should work without the language model running.

wiki ask "Where is the workshop?" --mode local: give a neutral, standalone answer with citations, or report insufficient evidence. Do not import chat history or assistant personality.

wiki chat: start the personal assistant, discuss ideas, and follow up naturally. The harness uses conversation context and calls retrieval only when needed.

wiki --help — explain commands, configuration, and required inputs. Show useful errors for missing files or an unavailable local model.

Save Your Results

Store actual outputs so a reader can inspect your project without rerunning the model. Label chat, ask, or search separately from local/online execution. Do not replace a failed result with an invented successful one.

- Save your model/runtime settings, source catalog, wiki pages, retrieval outputs, answers, citations, four ask-mode evidence cards, and the chat/search mode checks.
- Record errors or incomplete runs clearly. If you fix a problem, rerun the relevant checks and explain what changed.
- Include a transcript and recording or screenshots of the offline demonstration. Model weights can remain local; document their official download source and exact identifier.

README Requirements

The README is the grading entry point. A reader should understand what you built, how to run it, and what the evidence shows.

- Purpose and sources: describe the wiki's intended use, list your sources, and explain how originals connect to generated pages.
- Setup and device: list OS, CPU/chip, total RAM, GPU and dedicated VRAM or unified memory, available memory and free disk space. Include dependencies, model/version, quantization, runtime, downloads, and exact CLI commands. Explain the choice and report measured memory use and response time for local ingestion and an answer.
- Architecture: distinguish the model, retrieval tool, RAG workflow, CLI, and harness. Trace how the harness chooses a mode, manages conversation context, decides whether to retrieve, builds prompts, calls Gemma, checks citations, handles errors, and saves outputs.
- Design choices: explain passage size, retrieval method, research rules, assistant personality, context limits, and when chat retrieves notes. Explain the note naming and folder choices, how source IDs map to readable pages, and how re-ingestion avoids duplicates. Explain any model settings that affected results.
- Evidence: link the four ask-mode test records, chat/search mode checks, and offline recording or screenshots. Identify the model and data used for the run.
- Reflection: describe one real failure or limitation, what caused it if known, and one concrete improvement you would try.
- Optional online mode: explain how to select it, which hosted Gemma endpoint it uses, and what data it sends. Keep the required offline instructions separate and complete.

Suggested Workflow

1. Inspect your computer's specs first. Create your own project folder, install a compatible local runtime, and download a suitable quantized Gemma model. There is no starter repository to clone.
2. Choose your sources and write the four test questions with expected evidence before implementing retrieval.
3. Build ingestion with local Gemma, review the wiki pages, give them short descriptive filenames, add source references and meaningful links, and maintain a topic-organized index. Check navigation and graph labels in Obsidian, then ingest the same source again to check for duplicate notes.
4. Build chat, ask, search, ingest, and help. Connect conversation context, optional retrieval in chat, factual RAG in ask, prompts, local Gemma calls, citations, errors, and evidence logging through your own harness.
5. Disconnect from the internet, restart your CLI in local mode, ingest a local source, and run all four ask-mode questions plus the chat/search mode checks. Capture the terminal commands and actual results.
6. Write your README, inspect the submission as another reader would, and submit the single project. Add an online mode only if you want the extension.

Starter Prompt for Your AI Assistant

Help me build Assignment 4 for Class 5: a personal wiki CLI using a local open-weight Gemma model and RAG. I must create my own CLI and harness, not clone a sample app. First ask about my OS, CPU/chip, RAM, GPU/VRAM or unified memory, available memory, free disk space, and sources. Explain E2B, E4B, and 26B A4B MoE and help me choose one quantized model for my actual device. MoE's active parameter count does not equal its memory footprint. I do not need to run all three or buy hardware. Help me implement chat, ask, search, ingest, and help. Chat is a personal assistant with personality and conversation context; it should handle casual questions and drafting without forcing a notes lookup. Ask gives standalone, neutral factual answers from retrieved evidence, with citations or an insufficient-evidence response. Search returns original passages and paths without generating an answer. Explain the difference between the retrieval tool, the RAG workflow, and the harness that manages modes, context, tools, prompts, model calls, citations, errors, and saved outputs. Build a wiki for a human browsing Obsidian as well as for LLM retrieval. Use short descriptive Markdown filenames such as GPU Parallel Training.md, matching headings, a few topic folders, a grouped index.md, meaningful internal links, and traceable sources. Keep machine IDs in metadata and retrieval chunks outside the vault. Do not name notes after export tasks, hashes, or full sentences. Open vault/ in Obsidian and check actual filenames, graph labels, links, and source references; aliases alone are not a naming fix. If cleaning up existing notes, preserve originals, update incoming links and retrieval paths, rebuild the index, and rerun the question tests. Re-ingestion must not create duplicates or restore machine-style names. Keep assistant instructions separate from research rules. Use existing libraries or a compatible runtime where useful and explain the connecting code. Keep originals unchanged and instructions explicit. Define three answerable ask-mode questions and one unsupported ask-mode question. Also check casual chat, a conversational follow-up, raw search, and separation of ask from chat history. Measure memory use and response time with my wiki. After setup, disconnect the internet, restart the CLI, ingest a source, and run all tests without cloud services or fallback. Do not invent results. Online mode is optional; local mode is the default and must work independently.

Evidence Required in the README

- Include readable Obsidian screenshots of your personal memory: (1) an open note with a short descriptive filename, matching heading, source references, and related-note links; (2) the topic-organized page list or index; and (3) a graph view with readable note labels and meaningful connections. Filter to curated notes and hide attachments if needed; record the filter used. Show the source catalog and trace one note through a related note and back to its original evidence. Confirm links work and re-ingesting a source does not create duplicates. Omit or redact private information before sharing.
- Show the retrieved passages and source paths for each of the three answerable questions.

- Show the actual answers with citations, then explain whether the cited passages support the material claims.
- Show the unsupported question, retrieved context, and actual insufficient-evidence response. Document any initial failure and fix.
- Show CLI ingestion, all four ask-mode tests, and the chat/search mode checks after disconnecting the internet and restarting the CLI. Include commands, device specs, model/runtime identity, and saved evidence.
- Link your CLI and harness code and exact launch instructions. Show a real user running help, ingesting sources, chatting with a follow-up, searching original passages, and asking factual questions that receive local Gemma answers.

Submission Checklist

1. Create your own public GitHub repository containing your CLI and harness code, README, shareable wiki sources and pages, and test evidence. This is your submission repository, not a supplied sample.
2. Follow your documented setup and launch steps once more. Confirm required downloads are complete before disconnecting and running the offline demonstration.
3. Open your repository signed out and confirm the README, code, wiki, and evidence links are accessible.
4. Do not commit model weights or credentials. Provide the model's official download instructions and exact identifier, and use shareable material for the submitted wiki.
5. Submit your own project repository through the course submission portal.

Learning Focus

The purpose is to understand how a local model becomes useful with your own data. Explain what the harness does, why retrieval matters, how citations can be checked, and why missing evidence should lead to an honest limitation. RAG supplies context at inference time; it does not retrain Gemma.

Definition of Done

- A local open-weight Gemma model runs on your computer. Record the exact model/runtime, device specs, why it fits, and measured memory use and response time.
- At least three original sources are preserved. Generated wiki pages have short descriptive filenames, matching headings, source references, meaningful working links, and a topic-organized index. The page list and graph labels are readable in Obsidian. Re-ingesting a source does not create duplicate notes or machine-style names.
- Your own CLI exposes chat, ask, search, ingest, and help. The harness manages mode selection, assistant instructions, conversation context, optional retrieval, local Gemma,

citations, errors, and saved outputs. Chat handles ordinary conversation without demanding source evidence.

- Three answerable questions produce grounded, cited answers, and one unsupported question produces an insufficient-evidence response.
- CLI ingestion, all four ask-mode question tests, and chat/search mode checks run after the internet is disconnected and the CLI is restarted, without hosted embeddings, remote search, or cloud fallback.
- The README, source code, wiki, run instructions, offline evidence, and required Obsidian screenshots of your personal memory are complete and accessible.

Single Deliverable

- One public repository URL through the course submission portal, containing your own project and evidence. This finishes Assignment 4; it is not a phase of the capstone.

Scope

- Required: a local open-weight Gemma model suited to your device, your own CLI and harness with chat, ask, and search modes, your own small wiki, local retrieval, factual source citations, and an offline demonstration.
- A working CLI is required. A notebook or web interface is optional. Existing inference runtimes and libraries are allowed; model training, MCP, agents, and public deployment are not required.
- Optional: an online mode using a hosted Gemma endpoint through the same harness. Label the mode clearly and keep local mode as the default.
- The complete local workflow must work independently after setup. Online features never replace the required offline project.